makemore

Character-level language models in PyTorch: a bigram model and an MLP, trained to generate new name-like strings.

Overview

This repo contains my work on character-level language modeling. Given a dataset of names, the models learn the statistics of which character tends to follow which, then sample new names that look plausible but are not in the training set. It covers two models: a count-based/neural bigram model and a multilayer perceptron (MLP) that uses a longer context window.

Contents

- **Bigram model**: predicts the next character from only the previous one, built first from a count table and then as a single-layer neural network trained with gradient descent.
- **MLP model**: embeds a context of several previous characters, passes them through a hidden layer, and predicts the next character.
- **Data**: a plain text file of names, one per line (`names.txt`).

Models

1. Bigram

- Characters are mapped to integer indices, with a special start/end token `.`.
- Count version: build a 27x27 matrix of bigram counts, normalize rows into probabilities, and sample from them.
- Neural version: one-hot encode the input, multiply by a weight matrix, apply softmax, and minimize negative log likelihood. Converges to the same solution as the counts.
- Smoothing is applied to avoid zero probabilities.

2. MLP

- A block size sets how many previous characters are used as context.
- Each character index is looked up in a learned embedding table; embeddings are concatenated.
- A hidden layer with tanh nonlinearity feeds an output layer of logits over the vocabulary.
- Trained with cross-entropy loss and minibatch gradient descent, with learning rate tuning and decay.
- Data is split into train / dev / test sets to monitor overfitting.

Setup

```
git clone <your-repo-url>
cd makemore
pip install torch matplotlib jupyter
```

Usage

Open the notebooks and run the cells in order:

```
jupyter notebook
```

Each notebook trains its model and prints sampled names at the end.

Project structure

```
makemore/
  names.txt          # training data
  bigram.ipynb       # bigram model
  mlp.ipynb          # MLP model
  README.pdf
```

Results

Model	Context	Notes
Bigram	1 character	Baseline; samples are name-like but often awkward
MLP	Multiple characters	Lower loss and noticeably more realistic samples

Add your own final train/dev loss numbers here.

Ideas for next steps

- Tune embedding size, hidden size, and block size.
- Add batch normalization.
- Try deeper networks or recurrent / transformer models.